MediGuide AI

AI-Powered Medical Symptom Assessment and Patient Guidance Assistant

A LangChain + Streamlit Programming Assignment

Course Module: Building LLM Applications with LangChain

IMPORTANT MEDICAL & SAFETY NOTICE

The application built in this assignment is an **educational AI prototype only**. It must clearly state that it is **NOT** a replacement for a licensed doctor, professional diagnosis, emergency service, or medical treatment. It must never present a confirmed diagnosis. Every screen must direct users to consult a qualified healthcare professional and to seek emergency help in urgent situations.

Total marks: 100 • Suggested time: 1–2 weeks • Submission: GitHub repository + demo

1. Assignment Title

"AI-Powered Medical Symptom Assessment and Patient Guidance Assistant" — internally code-named **MediGuide AI**.

You will build an intelligent Streamlit application where a user enters basic information and symptoms, and a LangChain-powered LLM generates structured, safety-focused preliminary guidance.

2. Real-World Problem Scenario

A small healthcare-technology startup wants to build a prototype called **MediGuide AI**. Patients often search random websites when they experience symptoms. This produces unreliable information, anxiety, and confusion.

The company wants a responsible prototype that:

- Accepts patient information (age, gender).
- Accepts multiple symptoms, their duration, and a severity level.
- Accepts additional medical context (existing conditions, medications, notes).
- Uses LangChain and an OpenAI LLM to analyse the information.
- Produces structured, understandable guidance.
- Highlights a possible urgency level and recommends next steps.
- Displays prominent safety warnings and never claims a confirmed diagnosis.

3. Learning Objectives

By completing this assignment you will be able to:

- Integrate an OpenAI chat model into a Python app using **ChatOpenAI**.
- Design reusable prompts with **PromptTemplate** and **ChatPromptTemplate**.
- Work directly with **SystemMessage**, **HumanMessage**, and **AIMessage**.
- Force a model to return **structured JSON** and parse it safely.
- Build a reusable pipeline with **LLMChain**.
- Implement **streaming** output for a responsive user experience.
- Apply **in-memory** and **SQLite** caching to reduce cost and latency.
- Build a professional, safety-first **Streamlit** interface.

4. Technologies Required

Layer	Technology
Language	Python 3.10+
LLM framework	LangChain (langchain, langchain-openai, langchain-community, langchain-core)
Model provider	OpenAI (e.g. gpt-4o-mini)
User interface	Streamlit

Secrets	python-dotenv (.env file)
Caching	InMemoryCache + SQLiteCache

5. Functional Requirements

1. The app collects age, gender, symptoms, duration, severity, existing conditions, medications, and notes.
2. On submit, the app builds a prompt from the inputs and sends it to the LLM via an LLMChain.
3. The LLM returns a JSON object with summary, possible conditions, urgency level, next steps, doctor questions, and warning signs.
4. The app parses the JSON safely and renders a results dashboard.
5. A human-readable narrative is streamed live into the UI.
6. The urgency level is shown with an appropriate colour (LOW/MEDIUM/HIGH/EMERGENCY).
7. Caching (in-memory or SQLite) can be switched on to speed up repeated identical requests.
8. A medical disclaimer is visible on every relevant screen.

6. Non-Functional Requirements

- **Safety:** disclaimers must be impossible to miss; no confirmed diagnosis.
- **Reliability:** invalid JSON must never crash the app.
- **Modularity:** backend logic lives in `src/`, UI in `app.py`.
- **Readability:** beginner-friendly comments throughout.
- **Security:** the API key is loaded from `.env`, never hard-coded.
- **Performance:** caching reduces repeated API calls.

7. LangChain Concepts You Must Implement

Concept	Requirement
ChatOpenAI	Integrate the OpenAI chat model.
System / Human / AI messages	Define the role + safety rules and send patient data; show how AIMessage fits a conversation.
PromptTemplate	A reusable single-string template with variables (age, gender, symptoms, ...).
ChatPromptTemplate	A System + Human conversation carrying the safety rules and patient data.
Structured JSON	Instruct the model to return the exact JSON schema; parse it safely.
LLMChain	At least one reusable chain for the assessment.
Streaming	Stream the final guidance using .stream() into a Streamlit component.
Caching	Demonstrate BOTH InMemoryCache and SQLiteCache and explain the difference.

8. Required Streamlit Features

Sidebar: app name, description, medical disclaimer, model configuration, language selection.

Main form (use these widgets): `text_input, text_area, selectbox, multiselect, slider, button`.

Results dashboard (use these display elements): `st.metric, st.warning, st.info, st.error, st.success, st.expander`, plus tabs and/or columns, and `st.write_stream` for streaming.

9. Input Requirements

- Patient age (text_input).
- Gender (selectbox).
- Symptoms (multiselect + optional free-text).
- Duration of symptoms (selectbox).
- Severity 1–10 (slider).
- Existing medical conditions (text_area).
- Current medications (text_area).
- Additional notes (text_area).
- Answer language (selectbox).

10. Expected Output

The dashboard must display: (1) a patient symptom summary, (2) AI-generated general information, (3) possible conditions for education only, (4) an urgency level, (5) recommended next steps, (6) questions to ask a healthcare professional, and (7) warning signs requiring immediate attention.

The model must return valid JSON with this exact structure:

```
{
  "summary": "",
  "possible_conditions": [ { "name": "", "reason": "" } ],
  "urgency_level": "",
  "recommended_next_steps": [],
  "questions_for_doctor": [],
  "warning_signs": []
}
```

11. Suggested Project Architecture

```
medical_ai_assistant/
|
|-- app.py                (Streamlit UI - run this)
|-- requirements.txt
|-- .env.example
|-- README.md
|-- src/
|    |-- __init__.py
|    |-- config.py        (settings + form options)
|    |-- prompts.py       (PromptTemplate + ChatPromptTemplate + JSON schema)
|    |-- chains.py        (ChatOpenAI, LLMChain, streaming)
|    |-- cache_manager.py (in-memory + SQLite caching)
|    |-- utils.py         (safe JSON parsing + helpers)
|
```

```
|-- docs/
     |-- Medical_AI_Assignment.pdf
```

Data flow: form input → build prompt inputs → select cache → build LLM + LLMChain → run chain → parse JSON → stream narrative → render dashboard.

12. Step-by-Step Implementation Tasks

1. Set up the project folder, virtual environment, and install requirements.
2. Create `config.py` to load the API key and hold form options.
3. Write the system prompt and the JSON schema instruction in `prompts.py`.
4. Build a **PromptTemplate** and a **ChatPromptTemplate** using the patient variables.
5. In `chains.py`, build **ChatOpenAI** and a reusable **LLMChain**.
6. Add a raw System/Human/AI message demo function.
7. Add a streaming generator using `llm.stream()`.
8. In `cache_manager.py`, implement in-memory and SQLite caching switches.
9. In `utils.py`, implement **safe JSON parsing** and helpers.
10. Build the Streamlit sidebar, form, and results dashboard in `app.py`.
11. Wire streaming into `st.write_stream` and render the JSON safely.
12. Add disclaimers everywhere and test the scenarios in section 18.

13. API Key Setup Instructions

1. Create an account at platform.openai.com and generate a key under API keys.
2. Copy `.env.example` to `.env`.
3. Paste your key: `OPENAI_API_KEY=sk-...`
4. Never commit `.env` to version control (add it to .gitignore).
5. The app reads the key with `python-dotenv` at startup.

14. Caching Requirements

You must demonstrate **both** cache types and explain the difference in your README:

	InMemoryCache	SQLiteCache
Stored in	RAM (memory)	A file on disk (.db)
Speed	Fastest	Fast, slightly slower
Survives restart?	No	Yes
Best for	One session	Reusing across sessions

Register a cache once with `set_llm_cache(...)`; LangChain then checks it automatically before every model call. Submitting the same form twice should be visibly faster the second time.

15. Streaming Requirements

Use the model's `.stream()` method to yield chunks of the guidance narrative, and display them live with `st.write_stream()` so the user sees a natural typing effect instead of waiting for the whole answer.

```
def stream_narrative(llm, inputs):
    messages = NARRATIVE_CHAT_TEMPLATE.format_messages(**inputs)
    for chunk in llm.stream(messages):
        if chunk.content:
            yield chunk.content   # st.write_stream prints each piece
```

16. JSON Response Requirements

- Instruct the model to return ONLY valid JSON matching the schema in section 10.
- Strip any accidental ```json fences or surrounding text before parsing.
- Use try/except around `json.loads()`; never let bad JSON crash the app.
- On failure, show a friendly message and the raw output for debugging.

17. Medical Safety and Disclaimer Requirements

Non-negotiable safety rules:

• The app is clearly labelled an **educational AI system**, not a doctor.
• It must **never** present a confirmed diagnosis.
• A disclaimer appears in the sidebar, the main area, and the results.
• EMERGENCY-level output must tell the user to seek emergency help immediately.
• The system prompt encodes these constraints so the model behaves safely.

18. Testing Scenarios

#	Input	Expected behaviour
1	Age 25, runny nose + sore throat, 1-3 days, severity 2	Urgency LOW; calm monitoring advice.
2	Age 40, fever + cough, 4-7 days, severity 6	Urgency MEDIUM/HIGH; advises seeing a professional.
3	Severe chest pain + shortness of breath	Urgency HIGH/EMERGENCY; urges immediate help.
4	Submit the same form twice (cache on)	Second run is faster; identical result.
5	Empty symptoms	App warns the user and does not call the API.
6	Language = Urdu	Guidance text returns in Urdu.

19. Evaluation Rubric (100 marks)

Criterion	Marks
Streamlit UI/UX	15

ChatOpenAI integration	10
PromptTemplate implementation	10
ChatPromptTemplate and messages	10
LLMChain implementation	10
Structured JSON output	10
Streaming implementation	10
Caching implementation	10
Code quality and documentation	10
Testing and creativity	5
TOTAL	**100**

20. Bonus Features (optional)

- Conversation history / patient session history.
- Download the guidance as a PDF report.
- Multiple language support (beyond the required set).
- Dark / light mode toggle.
- Voice symptom input.
- A small analytics dashboard (urgency distribution).
- Docker deployment.

21. Submission Requirements

1. A public or shared GitHub repository containing every file in the structure above.
2. A working `requirements.txt` and a `.env.example` (never your real key).
3. A README with setup, run instructions, and your caching explanation.
4. A short screen recording or live demo showing the streaming output and dashboard.
5. Well-commented, modular code that runs with `streamlit run app.py`.

Reminder: this project is for education only. It is not a medical device and must not be used for real diagnosis or treatment.